# Derivation D1: Heap Purge Policy Amortized Bound

Accounting for Hash Set Dynamics and Resolving Fixed-Bucket Cost

**Swarna (250001070)** – Role P1 (Heap Engineer)
*CS 253 / MA 253 Data Structures & Algorithms Lab*

21 November 2026

## 1 Problem Statement and System Model

Structure A implements an event queue using a binary min-heap keyed by the composite lexicographical tuple $(t(e), s(e))$, representing timestamp and monotonic sequence number. In a discrete-event network simulator modeling packet transmissions with retransmission timeouts, acknowledging a packet often cancels several pending timeouts. To prevent immediate expensive $O(P)$ linear scans or heap searches, cancellations are handled lazily: when cancel($e$) is called, event $e$'s unique sequence number is inserted into a hash set without extracting the node immediately.

The heap physically stores $P$ entries, partitioned into $L$ live events and $C$ cancelled events:

$$P = L + C \tag{1}$$

The purge policy triggers when cancelled entries strictly outnumber live entries:

$$C > L \tag{2}$$

Upon triggering, the heap is compacted by discarding all cancelled entries and reconstructing a valid min-heap from the remaining $L$ live entries using Floyd's bottom-up linear-time heap construction algorithm. Our objective is to prove that under this policy, the amortized cost per cancellation is $O(1)$, and the amortized costs of insert and deleteMin are $O(\log L)$.

## 2 Resolution of the Fixed-Bucket Clearing Cost

**Audit Finding (Legacy Prototype Bottleneck):**
The legacy prototype implemented `id_set` using a fixed array of $B = 100{,}000$ buckets. On every purge, `id_set::clear()` traversed all 100,000 buckets to reset linked chains, taking $\Theta(B)$ time regardless of the actual live size $L$. In simulation runs where $L \approx 250$, a purge was triggered every $\sim 250$ cancellations, requiring 100,000 bucket operations. The resulting clearing overhead of $\frac{100{,}000}{250} = 400$ operations per cancellation dominated execution time, rendering the heap 5.8× slower than the AVL tree and violating the theoretical amortized bound.

**Resolution in Structure A:**
We replaced the fixed-bucket structure with `CustomHashSet`, a from-scratch open-addressing hash set utilizing linear probing, dynamic power-of-two table resizing (load factor $\alpha \leq 0.60$), and backward-shift deletion (Knuth Algorithm R, leaving zero tombstones). Most importantly, an auxiliary dense array `occupied_indices_` tracks the exact table indices currently in use. When `clear()` is invoked, it iterates strictly over `occupied_indices_[0 ... C-1]`, resetting occupied slots in $\Theta(C)$ time. Because capacity is maintained proportional to $C$, the clearing cost is strictly $\Theta(C)$, completely eliminating fixed-size overhead.

# 3 Potential Method and Amortized Analysis

We analyze the sequence of operations using the Physicist's Potential Method. Define the potential function $\Phi$ at any state of the data structure as:

$$\Phi = 2 \cdot C \tag{3}$$

where $C$ is the number of cancelled events physically stored in the min-heap.

**Properties of $\Phi$:**

1. At initial construction and immediately after any complete purge, $C = 0 \implies \Phi = 0$.
2. Since $C \geq 0$ at all times, $\Phi \geq 0$ throughout all states. Thus, the total amortized cost is an upper bound on the total actual cost.

## 3.1 Case 1: Insert Operation (insert($e$))

An insertion appends event $e$ to the end of the heap array (size $P$) and sifts up, requiring at most $\lceil \log_2(P+1) \rceil$ key comparisons. The event is also tracked in the live set in $O(1)$ amortized time. Because the count of cancelled items $C$ is unchanged:

$$\Delta\Phi = 2C - 2C = 0 \tag{4}$$

The amortized cost is:

$$\widehat{c}_{\text{insert}} = c_{\text{actual}} + \Delta\Phi = O(\log P) + 0 = O(\log P) = O(\log L) \tag{5}$$

(since $P < 2L$ prior to any purge trigger).

## 3.2 Case 2: Cancel Operation Without Purge (cancel($e$))

When cancel($e$) is invoked and $C + 1 \leq L$, no purge occurs. The sequence number $s(e)$ is inserted into `CustomHashSet` in $O(1)$ expected time. The number of cancelled items increases from $C$ to $C + 1$.

$$\Delta\Phi = 2(C+1) - 2C = +2 \tag{6}$$

The amortized cost is:

$$\widehat{c}_{\text{cancel}} = c_{\text{actual}} + \Delta\Phi = O(1) + 2 = O(1) \tag{7}$$

## 3.3 Case 3: Cancel Operation With Purge (cancel($e$))

When cancel($e$) causes $C > L$, a purge is triggered immediately. Let $C_p$ denote the number of cancelled entries at the purge point. Because $C_p > L$ and $P = L + C_p$, we have:

$$C_p > \frac{P}{2} \implies P < 2C_p \quad \text{and} \quad L < C_p \tag{8}$$

The purge procedure executes:

1. **Filtering live elements:** Linear scan over heap array of size $P$, taking $\alpha_1 P$ time.
2. **Floyd's bottom-up `buildHeap`:** Sifting down on $L$ elements takes $\alpha_2 L$ time.

3. **Clearing `CustomHashSet`:** Iterating across $C_p$ occupied slots in `occupied_indices_` takes $\alpha_3 C_p$ time.

Total actual purge cost:

$$T_{\text{purge}} = \alpha_1 P + \alpha_2 L + \alpha_3 C_p \leq (2\alpha_1 + \alpha_2 + \alpha_3) C_p = K \cdot C_p \tag{9}$$

for some positive hardware constant $K$.

Following the purge, all cancelled events have been eliminated, so $C_{\text{after}} = 0$. The potential change is:

$$\Delta\Phi = 2 \cdot 0 - 2 \cdot C_p = -2C_p \tag{10}$$

Choosing the potential scaling factor such that each cancel deposit covers $\frac{K}{2}$ work, we obtain:

$$\widehat{c}_{\text{purge}} = T_{\text{purge}} + \Delta\Phi \leq K \cdot C_p - K \cdot C_p = 0 \tag{11}$$

Thus, the actual execution cost of purging is fully pre-paid by the potential stored during previous lazy cancellations.

### 3.4 Case 4: DeleteMin Operation (deleteMin($out$))

deleteMin inspects the root. If the root is live, it is extracted, the last element is moved to the root, and sifted down in $O(\log P)$ time ($\Delta\Phi = 0$). If the root was lazily cancelled, it is popped, decrementing $C$ by 1 ($\Delta\Phi = -2$). The released potential pays for the redundant sift-down. Amortized cost: $O(\log P) = O(\log L)$.

## 4 Conclusion

By eliminating the fixed $\Theta(100{,}000)$ clear bottleneck through dense index tracking in `CustomHashSet`, the heap purge policy guarantees:

1. Heap physical size satisfies $P \leq 2L$ at all steady states.
2. Amortized cost per cancellation is strictly $O(1)$.
3. Amortized costs for insert and deleteMin are strictly $O(\log L)$.